

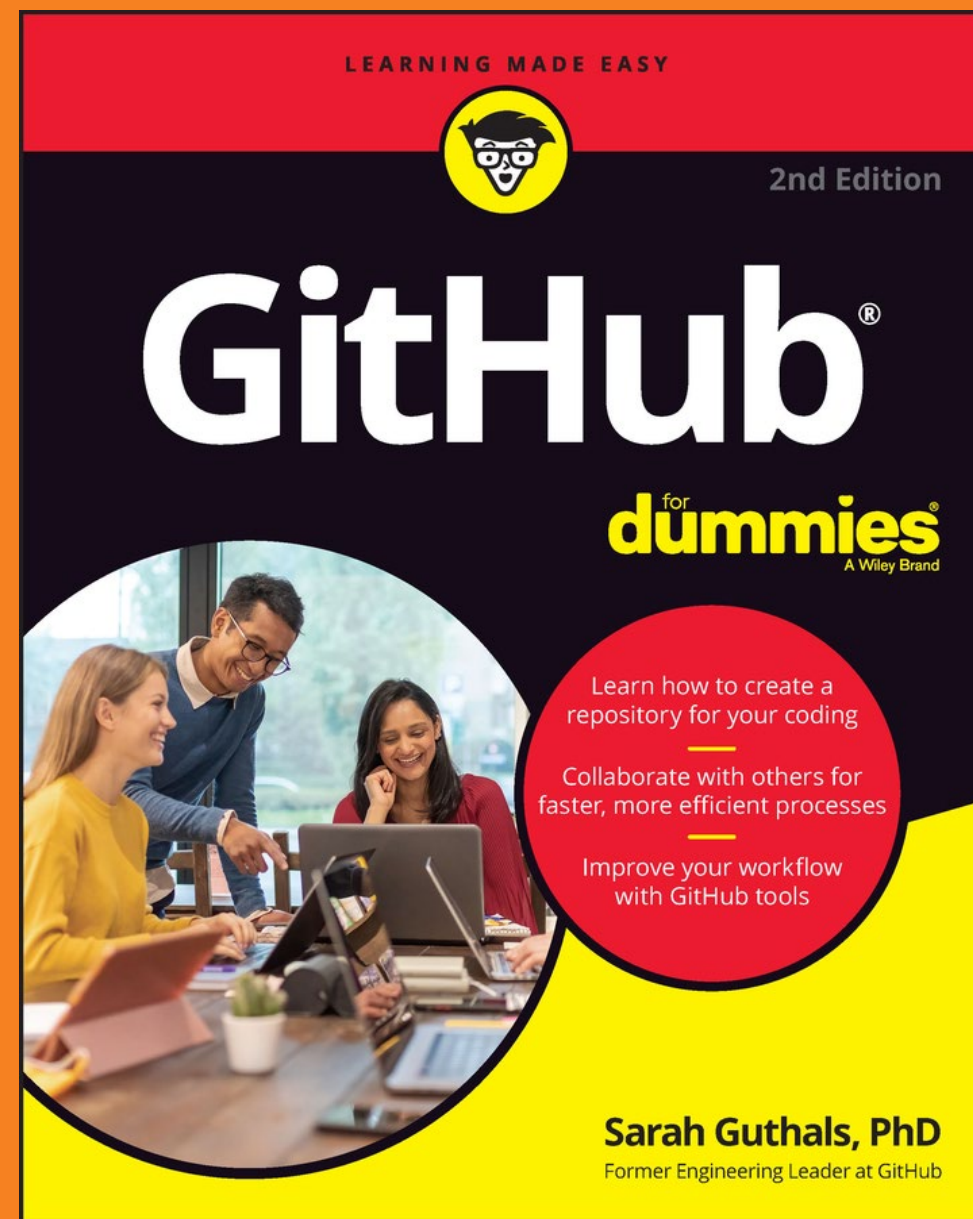
Source Code Management

Explore effective strategies and tools for optimizing source code management in software development.

CSV 201



L:3 T:0 P:0 Credits:3



► Textbooks:

GITHUB FOR DUMMIES by PHIL HAACK SARAH GUTHALS, WILEY

► References:

1. GIT POCKET GUIDE: A WORKING INTRODUCTION by RICHARD E. SILVERMAN, O'REILLY

2. EFFECTIVE DEVOPS: BUILDING A CULTURE OF COLLABORATION, AFFINITY, AND TOOLING AT SCALE by JENNIFER DAVIS, O'REILLY

Vision

To be a globally recognized school through excellence in teaching, learning and research for creating Computer Science professionals, leaders and entrepreneurs of future contributing to society and industry for sustainable growth.

Mission

- To build computational skills through hands-on and practice-based learning with measurable outcomes.
- To establish a strong connect with industry for in-demand technology driven curriculum.
- To build the infrastructure for meaningful research around societal problems.
- To nurture future leaders through research-infused education and lifelong learning.
- To create smart and ethical professionals and entrepreneurs who are recognized globally

Course Outcomes

- CO1 :: Understand DevOps and Software Delivery concepts
- CO2 :: Describe the Source Code Management and its benefits
- CO3 :: Study of Source Code Management Architecture and its types
- CO4 :: Understand different phases of Git and ability to transfer files into different phases
- CO5 :: Understand the managing the work with Git
- CO6 :: Understand how to Leveraging GitHub to collaborate on open-source projects

Skills To Be Learned

- Learn the importance of SCM in software development, including how it supports collaborative coding and ensures code integrity.
- Gain knowledge about popular SCM tools like Git, Subversion, Mercurial, and Perforce, and understand their key features and advantages.
- Learn key practices for effective source code management, such as regular commits, clear commit messages, and code reviews to maintain high standards in code quality and functionality.

Software Requirements



Certification/MOOC



- Certification: AWS CERTIFIED DEVOPS ENGINEER – PROFESSIONAL
- Platform: AMAZON WEB SERVICES
- [URL:https://aws.amazon.com/certification/certified-devops-engineer-professional/](https://aws.amazon.com/certification/certified-devops-engineer-professional/)
- Benefits: 1 CA

CSV 201: Source Code Management

CA category A0202

Exam Category 11 (Att. 5, CA 25, MTT 20, ETT 50)

Syllabus

Unit I

Typical Toolkit of DevOps : DevOps: An Overview, Achieving DevOps, Continuous Practices, How does CI Work?, Continuous Integration Practices, Commit Code Frequently, Maintain a Single Source Repository, Don't commit code, Keep the Build Fast, Every Commit Should Build the Mainline, Fix Broken Builds Immediately, Write Automated Developer Tests, Automated deployment helps, Benefits of Continuous Integration, Continuous Delivery, Continuous Delivery Process, Benefits of continuous delivery practices,

Unit II

Continuous Deployment and Version Control System : Brief about Continuous Deployment, Brief about Continuous Deployment Process, Benefits of Continuous Deployment, Version Control System (VCS), Repository and Working Copy, Types of version control systems, Benefits of Version Control Systems, Brief History of Version Control Systems, Basic Operations in a VCS, Examples of Version Control Systems, Concurrent Versions System (CVS), Subversion (SVN), Mercurial, Git, The Making - History of Linux and Git, Advantages of Git

Unit III

Centralized Version Control System vs Distributed Version Control System : Local Repository, Centralized Version Control System (CVCS), Distributed Version Control System (DVCS), Advantages of Distributed Version Control System , Private Workspace, Disadvantages of Distributed Version Control System , vs Distributed Version Control Systems, Comparison of CVCS and DVCS,

Unit IV

DVCS extended : Easier Merging, Easy to Scale Horizontally, Multiple Repositories Model, Multiple Repositories for Different Services, Resetting the Local Environment, Revert - Cancelling out the Changes

Unit V

Git : Introduction to Git, Data Structure used in Git for storing the versions, mostly used git commands with proper explanation like git add, git commit, git reset, git revert, git status, git config, git diff, git rebase, git log, git push, git pull, git fetch, git merge, git branch, git checkout, commit id, hash value generation in the commit id

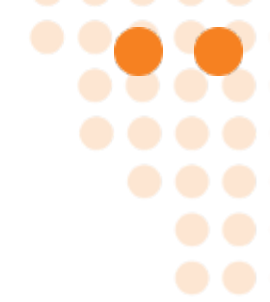
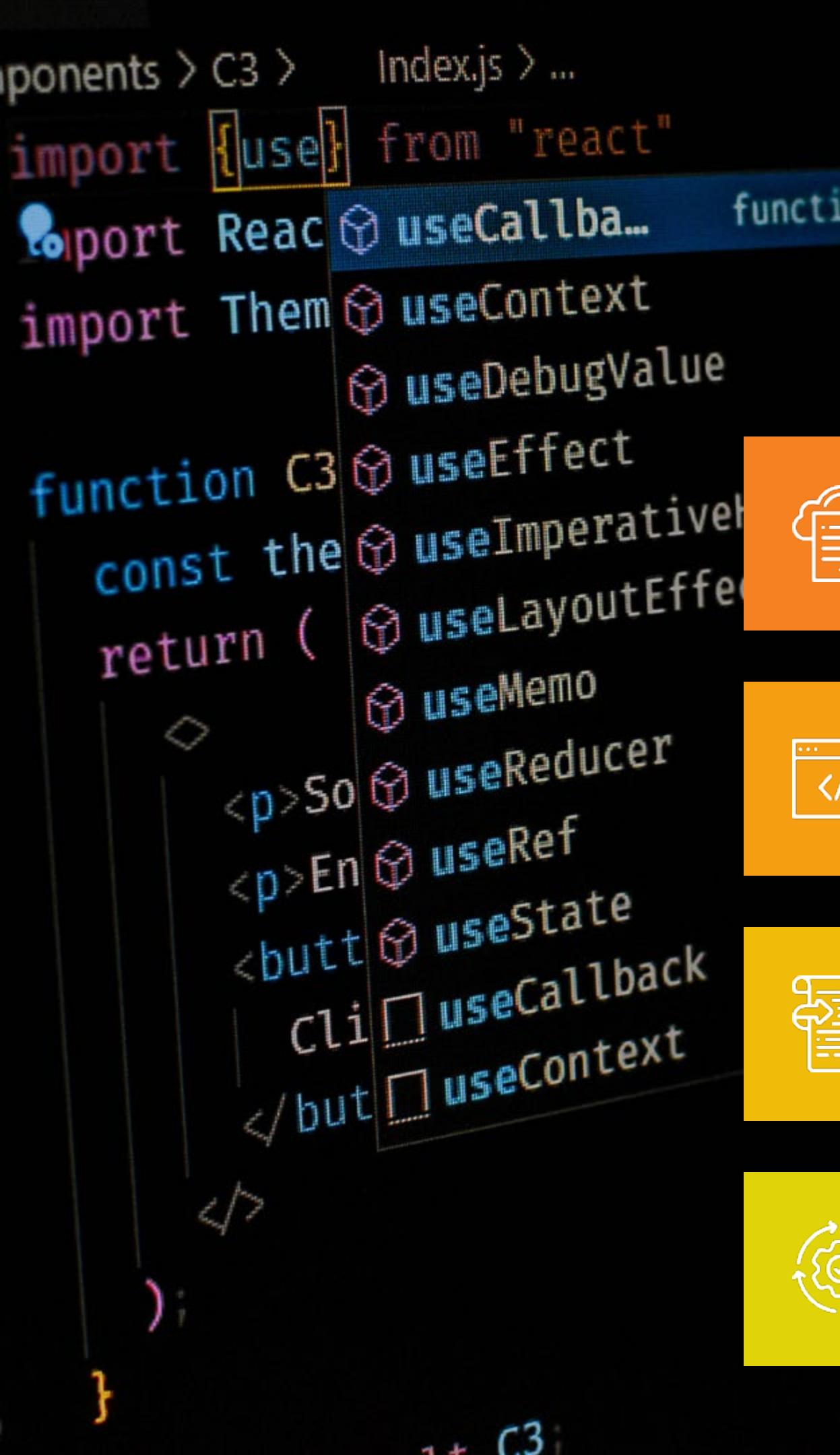
Unit VI

GitHub : Introduction to GitHub, Data Structure used in GitHub to store the Data, difference between git & GitHub, types of repositories on GitHub, pull requests on GitHub, forks on GitHub, cloning a repository from GitHub

Introduction to Source Code Management

Exploring the Essentials of SCM for Developers

01	Importance of SCM	02	SCM Tools	03	Best Practices in SCM	04	Version Control Systems
	Understanding the role of SCM in development.		Overview of popular tools used in SCM.		Key practices for effective source code management.		Exploring version control systems like Git.
05	Collaborative Development	06	Maintaining Code Integrity	07	Future Trends in SCM		
	How SCM supports collaborative coding.		Ensuring code integrity through SCM.		Emerging trends and technologies in SCM.		



Key Concepts of Source Code Management

Understanding Repositories, Branches, Commits, and Merges



Repositories

Storage locations for code.



Branches

Allow developers to work on different features simultaneously.



Commits

Snapshots of code changes.



Merges

Integrate changes from different branches.

Understanding Version Control Systems

Key Features and Types of VCS



Definition of

VCS manage changes to source code over time.



Purpose of

They provide versioning, branching, and merging.



Mechanisms of

Key features include tracking modifications.



Types of VCS

Centralized and distributed systems exist.



Advantages of Centralized VCS

Easier management but single point of failure.



Advantages of Distributed VCS

Better fault tolerance and offline capabilities.



Importance in

VCS are fundamental to efficient software control.

Git Workflow Steps

Key Steps for Effective Collaboration

01 Clone a Repository

Create a local copy for changes.

02 Create Feature Branches

Develop new features separately.

03 Make Commits

Record changes with messages.

04 Merge Branches

Integrate feature branches into main.

05 Push Updates

Upload local changes to remote.

Branching and Merging Strategies

Understanding strategies for efficient code management

01 Feature Branching

Utilizes separate branches for new features.

02 Git Flow

A structured branching model for releases.

03 Trunk-based Development

Continuous integration with a single branch.

04 Choosing the Right Strategy

Critical for team collaboration and efficiency.

05 Managing Code Changes

Essential for minimizing conflicts and errors.

06 Minimizing Integration Issues

Reduces risks of integration failures.

Key Features of GitHub

An Overview of GitHub's Capabilities

01

Popular platform for hosting Git repositories

GitHub is widely used for managing code projects.

02

Facilitates collaboration among developers

Supports teamwork through pull requests and reviews.

03

Integration with numerous tools

Offers compatibility with various development tools.

04

Support for open-source and private projects

Enables both open-source contributions and private work.

05

Vibrant community as a valuable resource

Active user community provides support and resources.

Best Practices in SCM

Ensuring Efficient and Reliable Code Management



Importance of Code Reviews in SCM

Enhancing Code Quality and Team Productivity

Vital Part of SCM Process

Code reviews are essential for effective SCM.

Promotes Code Quality

Ensures that code meets quality standards.

Facilitates Knowledge Sharing

Encourages team members to learn from each other.

Peer Reviews Before Integration

Changes are reviewed by peers prior to merging.

Provides Constructive Feedback

Helps developers understand and improve coding practices.

Ensures Adherence to Standards

Checks compliance with established coding standards.

Enhances Team Productivity

Streamlined processes lead to more efficient teamwork.

Effective Collaboration Tools

Utilization of tools that facilitate code reviews.

Techniques for Managing Repositories

Addressing Challenges in Performance and Merges



Challenges of Large Repositories

Large repositories can lead to slow performance.



Leveraging Large File Storage

LFS can improve handling of large files within repos.



Merging Complexities

Merging code in large repositories can become difficult.



Repository Splitting

Consider splitting repositories to enhance performance.



Using Submodules

Submodules can help manage dependencies effectively.

Backup and Recovery Strategies

Essential Approaches to Protect Codebase

Regular backups

Scheduled backups to safeguard codebase integrity.

Data replication

Ensures availability by duplicating data across locations.

Disaster recovery plans

Structured response plans for unexpected data loss.

Quick restoration

Swift recovery processes in case of hardware failures.

Maintaining project continuity

Ensures ongoing project success despite disruptions.

Best Practices in SCM Implementation

Key Insights and Lessons Learned

Effective Use of SCM Tools

Utilized SCM tools for process efficiency.

Streamlined Development Processes

Improved workflows to reduce delays.

Enhanced Collaboration

Facilitated better team communication.

Improved Code Quality

Incorporated best coding practices.

Successful Project Outcomes

Achieved goals through effective SCM.

Challenges in SCM

Identifying Key Issues in SCM Practices

Managing complex branching

Navigating multiple branches can lead to confusion.

Handling large codebases

Large repositories require efficient organization.

Ensuring security

Protecting code from unauthorized access is crucial.

Maintaining efficient collaboration

Teamwork is essential for successful SCM.

Addressing Supply Chain Management Challenges

Key Strategies for Success in SCM

Strategic Planning is Essential

Effective SCM requires well-thought-out strategies.

Adopting Agile Methodologies

Flexibility in processes enhances response time.

Leveraging Automation

Automation streamlines operations and reduces errors.

Implementing Robust Security Measures

Security protects data and builds trust in SCM.

Continuous Process Refinement

Regular updates improve efficiency and adaptability.

Benefits of Open Source SCM Tools

Key Highlights of Open Source Tools

01 Flexibility of Tools

Tools like Git and Mercurial allow customization.

02 Community-Driven Development

Development driven by active community contributions.

03 Cost-Effective Solutions

Open source tools provide low-cost code management.

04 Active Community Contributions

Communities enhance tools to keep them relevant.
